

Build systems that *outlive* everything about them.

A governance philosophy for long-lived software, an enterprise architecture framework, and a team-level development lifecycle — all designed for a world where AI assistants write most of the code and the systems we build today must survive the next decade of change.

[Read SEA 1.1 →](#)

[Explore the documents](#)

7

Documents across three tiers

Foundation · Operations · Supporting

32

Architectural gaps named

In current enterprise practice · 7 categories

14

Named failure modes

Every one a real pattern · every one preventable

CC

BY 4.0 Licensed

Adopt, adapt, cite the author

Four drift incidents.

Every principle traces to one.

These are not hypothetical failure modes. They happened during a real production build across five generations of AI models. Each incident revealed a specific missing control — and that control is now a named principle in SEA 1.1.

CD1

HIGH

Sealed Function Count Drift

The SSOT declared 19 sealed functions. The code had 23. Both were written by different AI sessions. Neither was wrong when written. Nobody noticed the delta for weeks.

→ Principle 1 · Contracts durable, implementations transient

CD2

CRITICAL

Regeneration Loss

The SSOT was "updated" by regeneration. Two sections silently disappeared. The AI had no way to know which sections were load-bearing. Caught only by a table-of-contents hash change.

→ Principle 2 · Intent captured, not just behaviour

CD3

CRITICAL

Wrong File Attribution

A bug fix was applied to MivaReelPlayer.kt. The bug persisted. The logic actually lived in ChapterPlayerScreen.kt — a file the AI had never opened. It described code instead of reading it.

→ Source-First rule · structural AI-readability

CD4

HIGH

Status Drift on Vulnerabilities

V1–V11 vulnerability fixes were applied to code. The SSOT still showed them as "Pending." Both correct when written. Mutually inconsistent for weeks — caught only during release readiness review.

→ Authority Chain · change-history as single source

Sealed Enterprise Architecture 1.1

The foundational whitepaper the rest of the methodology rests on. A new architectural philosophy for systems that must outlive their technologies, their teams, their owners — and sometimes even their original purpose.

SEA 1.1 · 63 PAGES · MAY 2026

The Bridge Problem for *software systems*.

A bridge commissioned in 1975 still stands today — despite its materials, engineers, owners, and the cities it connects all having changed. Civil engineering developed the discipline for long-lived structures. Software engineering has not. SEA 1.1 proposes what that discipline should look like.

[Read the whitepaper →](#)

[↓ PDF](#)

32 Gaps in current practice, across 7 categories

14 Named failure modes of long-lived systems

7 Principles that define SEA

5 Architectural layers in the framework

Seven documents. Three *tiers*.

Foundation establishes the philosophy. Operations specifies how to work under it. Supporting provides evidence, commercial tools, and developer companions. Each document is complete on its own; together they form a coherent methodology.

TIER 01 FOUNDATION — THE PHILOSOPHY EVERYTHING RESTS ON

FOUNDATIONAL

DOCUMENT 01 · SEA 1.1 · 63 PAGES

[Enterprise Architects](#) · [CIOs](#) · [CTOs](#)

Sealed Enterprise Architecture 1.1

32 gaps in current practice. 14 named failure modes. 7 principles. A 5-layer architecture for systems that must outlive everything about them.

[Read →](#)[↓ PDF](#)[↓ DOCX](#)

NEW · VOL 1

DOCUMENT 02 · HANDBOOK · 63 PAGES

[Platform Teams](#) · [Enterprise Ops](#)

SEA Implementation Handbook

Volume 1 — Measurement, Enforcement, Verification, Evolution. The operational companion that turns the philosophy into working practice.

[Read →](#)[↓ PDF](#)[↓ DOCX](#)

NEW · CURRENT

DOCUMENT 03 · SDLC V2.0 · 57 PAGES

[Tech Leads](#) · [Team Architects](#) · [Engineers](#)

The AI-Native SDLC v2.0

Team-level operations under SEA. Seven phases re-architected around sealed contracts, intent documents, and SEA-native governance. Full mode and team-autonomous mode. Supersedes v1.1.

Read →

↓ PDF

↓ DOCX

CURRENT

DOCUMENT 04 · MBG V1.1

[Developers](#) · [Individual Contributors](#)

Master Build Guide

The operational companion to the SDLC. Prompt library (PL-01 through PL-09), 12 failure modes, daily/weekly/milestone rituals, Model Selection Matrix.

Read →

↓ PDF

↓ DOCX

CURRENT

DOCUMENT 05 · CASE STUDY · V1.1

[Sceptics](#) · [Prospective Adopters](#)

From Solo to Production

The twelve-month MiVA build that produced the methodology. CD1–CD4 documented in detail. Five model generations survived. The evidence base.

Read →

↓ PDF

CURRENT · COMMERCIAL

DOCUMENT 06 · DRIFT AUDIT · V1.1

[CTOs](#) · [Engineering Leaders](#)

The Drift Audit

A standalone two-week diagnostic. Fixed-fee tiers (Starter, Standard, Extended, Retainer). Five reconciliation pairs. Severity classification. Qualifying disqualifiers.

Read →

↓ PDF

ARCHIVED · V1.1

DOCUMENT 07 · SDLC V1.1 · SUPERSEDED

[Reference only](#)

The AI-Native SDLC v1.1

The pre-SEA SDLC. Superseded by v2.0 in April 2026 via formal Major Version Bump. Preserved for teams currently on v1.1 who have not yet migrated.

Archival reflects SEA's own retirement-with-dignity principle — history preserved, not deleted.

Read archive →

↓ PDF

ARCHIVED · V1.0

DOCUMENT 08 · SEA 1.0 · SUPERSEDED

Reference only

Sealed Enterprise Architecture 1.0

The original SEA whitepaper, published April 2026. Superseded by SEA 1.1 in May 2026 via focused revision. Preserved as historical record of the framework's first articulation.

Archival reflects SEA's own retirement-with-dignity principle — history preserved, not deleted.

Read archive →

↓ PDF

— WHERE TO START

Seven documents. *Four reading paths.*

Reading every document is unnecessary for most readers. Four common roles, four suggested paths — each 2-4 documents that together give the reader what they need.



CTO / VP ENGINEERING

Deciding whether to adopt

- 1 SEA 1.1 — Parts I, II, IV
- 2 Handbook — Part I (Measurement)
- 3 Drift Audit — as commercial gateway



ENTERPRISE ARCHITECT

Integrating SEA into practice

- 1 SEA 1.1 — full document
- 2 Handbook — all four parts
- 3 SDLC v2.0 — Parts I, IV



TEAM / TECH LEAD

Adopting the team-level SDLC

- 1 SDLC v2.0 — full document
- 2 Master Build Guide — daily companion
- 3 SEA 1.1 — Part II (principles)
- 4 Case Study — if sceptical



SCEPTIC / EVALUATOR

Verifying the claims

- 1 Case Study — first; evidence base
- 2 Drift Audit — commercial maturity
- 3 SEA 1.1 — Annex A only

AUTHOR

Built from a real build. *Not a theory.*



Suneesh Sidharthan

Solution Architect · IT Project
Manager
AI Platform Engineering
Bengaluru, Karnataka, India

suneesh.sidharthan80@gmail.com

+91 9986898076

[LinkedIn ↗](#)

[GitHub ↗](#)

BACKGROUND

Over 20 years across telecom, banking, analytics, and digital platforms as a Solution Architect and IT Project Manager. This methodology emerged from building MiVA Education — a bilingual Android and PostgreSQL platform for Indian Class 10 students — almost entirely solo across five generations of AI assistants.

The frameworks were not designed in advance. They emerged from what broke — four drift incidents that each revealed a missing control. SEA 1.1 articulates the philosophy those patterns implied (originally published as 1.0 in April 2026; revised to 1.1 in May 2026). The SDLC v2.0 and Implementation Handbook operationalise it.

Available for Drift Audit engagements, methodology consulting, and SEA adoption support. Get in touch if your organisation is using AI in production and suspects the codebase no longer matches the story it tells about itself.

[Book a call](#)

[Explore Drift Audit](#)

Sealed Enterprise

Architecture 1.1

A foundational whitepaper. Thirty-two gaps in current practice. Fourteen named failure modes. Seven principles and a five-layer architecture for systems that must outlive their technologies, their teams, their owners, and sometimes even their original purpose.

VERSION	PAGES	AUDIENCE	LICENCE
1.1 · May 2026	63	Enterprise Architects · CIOs · CTOs	CC BY 4.0

[↓ Download PDF](#)

[↓ DOCX](#)

The Bridge Problem

A bridge commissioned in 1975 still stands today. Its concrete has been resurfaced three times. Its steel has been replaced. The original chief engineer retired decades ago. Ownership has changed three times. Even the towns it connects have transformed. And yet the bridge still works.

Civil engineering developed, over centuries, the implicit discipline of long-lived structures. Pillars are engineered to be *sealed* — they cannot be modified once poured. New loads are met by adding new pillars alongside, not by retrofitting the old. Software engineering has no such discipline. A codebase commissioned today will face every one of those same pressures over the next decade — and most current architectures have no formal answer for any of them.

SEA's core insight

Every long-lived system must explicitly separate what is *durable* from what is *transient*, and must provide named mechanisms for the transient to change without threatening the durable.

The 32 Gaps

SEA's Section 2 enumerates 32 assumptions in current architectural practice that were tolerable in the pre-AI era and have become lethal in the AI era. The four gaps that AI specifically created — **Stable AI model**, **Stable quality gates**, **Stable readers of documentation**, **Stable cognition** — are the reason this whitepaper exists.

CATEGORY	EXAMPLE GAPS	COUNT
Technology	Stable programming language · Stable framework · Stable AI model	6
Teams	Stable lead architect · Stable engineering team · Stable onboarding	4
Ownership	Stable corporate entity · Stable vendor relationships · Stable jurisdiction	5
Purpose	Stable user base · Stable business model · Stable regulatory regime	5
Verification	Stable definition of correct · Stable threat model · Stable quality gates	4
Knowledge	Stable readers of documentation · Stable decision log · Stable cognition	4
Evolution	Stable deprecation · Stable extension mechanism · Stable retirement ceremony	4

The 14 Failure Modes

When the gaps go unaddressed, systems fail in specific, recognisable ways. SEA names fourteen such patterns. Together they form a diagnostic vocabulary.

Ossification

System too rigid to change safely

Dissolution

System loses coherent identity

Ghost Ownership

Component has no current owner

Purpose Drift

Serves a purpose nobody uses

Amnesia

Original intent irretrievable

Vendor Capture

Cannot move off a specific vendor

Cognitive Debt

No human understands the whole

Inheritance Failure

Next generation cannot maintain it

Silent Contract Violation

Undocumented gotchas accumulate

Compliance Rot

Nominally compliant, actually not

Shadow Architecture

Real flows outside the diagram

Model-Induced Drift

AI eras leave archaeological strata

Plausibility Cascade

Plausible-but-wrong becomes load-bearing

Decommissioning Paralysis

Old systems never quite retire

The Seven Principles

These are axioms. All seven must hold for a system to be SEA-governed. Partial adoption produces partial protection.

P1

Contracts durable, implementations transient

A contract is a formal declaration, separate from the code that fulfils it. Implementations may be rewritten freely.

P2

Intent captured, not just behaviour

The intent document captures why a contract exists. Invariants, rejected alternatives, assumptions — all preserved.

P3**Succession is designed, not assumed**

Every contract has a named current Owner. Transitions are formal. No orphan contracts.

P4**Change is graduated, not binary**

Changes move through named stages: proposed, reviewed, experimental, parallel-live, adopted, retired.

P5**Purpose-change has its own protocol**

Major Version Bumps are ceremonial, rare, formal — the way bridges get replaced, not demolished.

P6**Portability is first-class**

Contracts are YAML. Intent documents are Markdown. They travel with the system across companies and stacks.

P7**AI-readability is structural**

Artefacts are structured for AI consumption. Future models can read them without training on your codebase.

The Five Layers

1**Contract Definition**

Sealed YAML contracts declaring what must persist. Language-agnostic. Immutable once sealed.

CONTRACT OWNER

2**Intent Capture**

Markdown intent documents paired with each contract. Why, not just what. Append-only.

CONTRACT OWNER

3**Verification**

Per-stack verifiers that prove implementations conform to contracts. Thin, replaceable.

PLATFORM TEAM

4

Governance

Roles, protocols, review cadence. Contract Owner, Reviewer, Sealed Review Board.

SEALED REVIEW BOARD

5

Evolution

Three mechanisms: extension, parallel-live replacement, Major Version Bump.

SEALED REVIEW BOARD

Relationship to Existing Frameworks

SEA does not replace TOGAF, DDD, Clean Architecture, or any other established practice. It provides the governance layer those frameworks have historically lacked.

EXISTING PRACTICE	HOW SEA INTEGRATES
TOGAF	Sealed contracts become first-class elements of the Architecture Continuum
Domain-Driven Design	Aggregate roots and bounded contexts map to sealed contract boundaries
Clean / Hexagonal	Ports are SEA sealed contracts. Adapters are transient implementations.
Microservices	Each service's public API becomes a sealed contract; internals stay transient
Spec-Driven Development	SDD generates specs; SEA seals the critical subset. Complementary.
SAFe Agile	Extension protocol fits sprint cadence; quarterly reviews align with Inspect & Adapt

CONTINUE READING

Download the full 63-page whitepaper for the complete specification, worked YAML contract example, governance protocol, and enterprise adoption pathway.

↓ Download PDF

Read the Handbook →

SEA Implementation

Handbook · Volume 1

The operational companion to SEA 1.1. Four dimensions covered in depth: Measurement, Enforcement, Verification, Evolution. First-draft design thinking applied rigorously — honest about what is grounded and what awaits adoption experience.

VERSION

Volume 1 · April 2026

PAGES

63

AUDIENCE

Platform Teams · Enterprise Ops

LICENCE

CC BY 4.0

[↓ Download PDF](#)[↓ DOCX](#)

What the Handbook Addresses

SEA 1.1 articulates the philosophy. The Handbook answers the operational questions that arise once an organisation has decided to adopt SEA — the questions a platform team asks during the first six months of implementation.

PART I — Measurement

How do we know SEA is working? Contract Integrity Score with six weighted sub-metrics. Portfolio indicators. Governance indicators. Leading vs lagging. Anti-metrics.

PART II — Enforcement

What physically prevents modification? Four-layer defence: repository, build pipeline, runtime, governance. CODEOWNERS, hash verification, breach response.

PART III — Verification

Beyond PASS/FAIL. Five compliance states (Conforming, With Warnings, Partial, Non-Conforming, Unverifiable). Four-level severity per invariant.

PART IV — Evolution

How sealed systems change without losing their sealed character. Extension catalogue. Parallel-live playbook. Rollback. MVB protocol. Retirement.

One honest note

The Handbook's operational guidance is first-draft design thinking, applied rigorously, but not battle-tested at enterprise scale. Volume 2 — covering team-scale operations, SDLC integration patterns, and reference templates — awaits early adoption experience before being written.

What Makes It Actionable

The Handbook specifies, not just recommends. The Contract Integrity Score is a six-sub-metric formula with specific weights:

SUB-METRIC	WHAT IT MEASURES	WEIGHT
Intent Completeness	Coverage of required intent sections	25%
Change History Cleanliness	Whether the log is current, sequential, attributed	20%
Invariant Coverage	Declared invariants with enforcement mechanisms	20%
Ownership Currency	Owner and Reviewer active in their roles	15%
Consumer Alignment	Known consumers on the current version	10%
Review Cadence Adherence	Quarterly reviews happening on schedule	10%

The Enforcement section includes a full CODEOWNERS template, recommended repository layout, Git tag patterns, and the specific build-pipeline gates that make sealing enforceable. The Verification section provides the YAML schema for declaring violation severity at the invariant level. The Evolution section includes a catalogue of four extension patterns with example YAML and four anti-patterns to refuse.

CONTINUE READING

63 pages with complete specifications, worked examples, YAML schemas,
and operational templates.

↓ Download PDF

Read SDLC v2.0 →

The AI-Native *SDLC v2.0*

A re-architected development lifecycle for teams using AI assistants within organisations that have adopted SEA 1.1. Seven phases rebuilt around SEA-native artefacts. Full mode and team-autonomous mode. This is a Major Version Bump; v1.1 is archived.

VERSION	PAGES	AUDIENCE	SUPERSEDES
2.0 · April 2026	57	Tech Leads · Team Architects	SDLC v1.1 (archived)

↓ Download PDF

↓ DOCX

The Seven Phases

The phase names carry forward from v1.1. What each phase produces and how it connects to governance is substantively re-architected around SEA's sealed contracts, intent documents, and governance protocol.

1

FRAME
Problem, core bet, non-goals, and — new in v2.0 — initial contract candidates. Before any solution.
SOCRATIC AI

2

SEAL
Full SEA sealing protocol. Sealed contracts with paired intent documents. External review required.
CONTRACT DRAFTER

3

SPEC

Extension proposals, verification requirements, consumer mapping, implementation plan organised around contracts.

SPEC AUTHOR

4

BUILD

Implementation against sealed contracts. Source-First enforced. One chat per unit.

IMPLEMENTATION PARTNER

5

VERIFY

Five compliance states instead of pass/fail. Severity classification per violation. Cross-family AI review.

INDEPENDENT REVIEWER

6

RECONCILE

Five reconciliation pairs. Weekly team rhythm. Feeds enterprise SRB quarterly review in full mode.

AUDITOR

7

EVOLVE

Extension, parallel-live replacement, or Major Version Bump. Sealed contracts are never edited.

EXTENSION DESIGNER

The Authority Chain

When two artefacts disagree, the leftmost wins. Declared once, enforced permanently. This is how v2.0 resolves conflicts between what the code shows, what the change history records, and what documentation claims.

1

DATABASE

The live state. Highest authority. What the database actually contains beats every document that describes it.

GROUND TRUTH

2

CHANGE HISTORY

Append-only log inside each sealed contract. Replaces v1.1 Decision Log for contract matters.

APPEND-ONLY

3

SEALED CONTRACT

IMMUTABLE

The YAML specification. Machine-readable. Cannot be edited once sealed.

4

CODE

TRANSIENT

Must conform to the contract. If code disagrees with contract, code is wrong (or it's an EVOLVE event).

5

DOCS

DERIVED

Narrative explanations. Derived view. If docs disagree with anything above, docs are out of date.

Two Operating Modes

Full mode assumes SEA governance is operational at enterprise level. Team-autonomous mode supports teams using SDLC v2.0 without requiring organisational SEA adoption first. Both produce SEA-compatible artefacts; the difference is where the governance authority resides.

GOVERNANCE ROLE	FULL MODE	TEAM-AUTONOMOUS MODE
Contract Owner	Named individual, potentially outside the team	Senior team member
Contract Reviewer	Separate named individual, formally accountable	Another senior team member
Sealed Review Board	Standing enterprise body, 4-8 members	The team itself, quarterly cadence
Extension approval	Owner + Reviewer; SRB escalation when cross-team	Team lead + senior engineer
MVB approval	SRB unanimous + executive sponsor	Team leadership + external stakeholder
Drift log	Feeds enterprise log	Team-local, preserved for future integration

GOVERNANCE ROLE	FULL MODE	TEAM-AUTONOMOUS MODE
Verifier tooling	Enterprise platform team	Team-built or off-the-shelf

Five Compliance States

Replaces v1.1's binary pass/fail. Each contract-implementation pair lands in one of five states, with implications for whether deployment proceeds.

STATE	GATE BEHAVIOUR	NEXT ACTION
Conforming	Passes	Unit is done; merge to main
Conforming with Warnings	Passes with documented warnings	Address in next regular cycle
Partial	Blocks merge	Return to BUILD; specific issues documented
Non-Conforming	Blocks merge; serious concern	Escalate to contract-level review
Unverifiable	Blocks merge; infrastructure concern	Verifier or contract investigation

Five Reconciliation Pairs

Weekly in RECONCILE. Each pair catches a distinct class of drift.

A Code ↔ Contract Missing operations, signature drift, invariant violations	B Change History ↔ Code Ungoverned modifications to sealed contracts
C DB Schema ↔ Queries Column rename drift, type drift, missing columns	D Sealed baseline ↔ current files Contract files modified outside the extension protocol



Intent ↔ Contract

Contract evolution not reflected back into intent. New in v2.0.

RACI — AI as a Role

AI-Builder and AI-Reviewer are different chat sessions, ideally from different model families. Cross-family review catches failures same-family review systematically misses.

PHASE	RESPONSIBLE	ACCOUNTABLE	CONSULTED	INFORMED
FRAME	Human lead	Human lead	AI (Socratic)	Team
SEAL	AI-Builder + Human	Contract Owner	AI-Reviewer (different family)	SRB (full mode)
SPEC	AI-Builder	Human lead	AI-Reviewer	Contract Owner
BUILD	AI-Builder + Human	Human	—	—
VERIFY	AI-Reviewer (different family)	Human	—	Contract Owner
RECONCILE	AI (auditor mode)	Human lead	—	SRB (full mode)
EVOLVE	AI-Builder	Contract Owner	AI-Reviewer	SRB (full mode)

What Transfers From v1.1

A team comfortable with v1.1 will find most habits still operate. The seven-phase structure remains. "Ready for external review" remains the load-bearing phrase. One-chat-per-unit, Source-First, cross-family AI review, model-agnostic-by-default — all unchanged.

What changes is what the phases produce and where their outputs are governed. The SSOT becomes a contract + intent pair. The Decision Log for contract matters becomes the change

history inside the contract. The Feature Checklist becomes the Extension Proposal. The three-tier L1/L2/L3 verify becomes five compliance states with severity classification.

Should you migrate from v1.1?

Migrate if: your organisation has adopted SEA, your team has grown past 15 engineers, you have entered regulated territory, or drift incidents have traced to governance gaps.

Stay on v1.1 if: you are under 5 engineers with a short horizon, your organisation has no SEA ambition, your team is change-fatigued, or you are mid-way through a major v1.1 implementation. Both versions remain legitimate choices.

CONTINUE READING

57 pages with detailed phase specifications, artefact templates, migration path from v1.1, and CD1–CD4 reframed through SEA.

↓ Download PDF

Read Master Build Guide →

Master Build

Guide v1.1

The operational companion that turns the SDLC from framework into daily practice. Prompt library, failure modes catalogue, rituals, Model Selection Matrix. Written for the developer who lives in BUILD chats.

VERSION	AUDIENCE	LICENCE
1.1 · April 2026	Developers · ICs	CC BY 4.0

↓ Download PDF

↓ DOCX

Model Selection Matrix

Task types are stable. The "Current Pick" column is the only thing that changes when a new model releases.

TASK	CRITERIA	CURRENT PICK (APRIL 2026)
SEAL — contract drafting	Deep reasoning, schema fluency	Claude Opus 4.7
SPEC — specification	Rigor, consistency across long context	Claude Opus 4.7
BUILD — routine implementation	Fast, accurate code synthesis	Claude Sonnet 4.6
BUILD — complex algorithms	Deep reasoning over subtle invariants	Claude Opus 4.7

TASK	CRITERIA	CURRENT PICK (APRIL 2026)
VERIFY — cross-family review	Independent perspective	GPT-5 / Gemini 3 Pro
RECONCILE — drift audit	Thoroughness, pattern recognition	Claude Opus 4.7
FRAME — Socratic clarity	Question-asking, assumption-surfacing	Claude Opus 4.7

Nine Prompt Templates

A prompt is not chat text — it is reusable infrastructure encoding hard-won patterns. Every prompt has a phase, a model class, and a purpose.

PL-01	FRAME Distillation Converts raw product ideas into Problem + Core Bet + Non-Goals + contract candidates	PL-02	Change History Entry Produces properly structured entries for a contract's change history
PL-03	Surgical Edit Modifies one function, one file, with explicit no-regression guarantee	PL-04	Ship-Ready Check Pre-VERIFY self-audit covering all five compliance states
PL-05	Source-First Enforcement Forces the AI to view actual files before describing them	PL-06	Reality Check Challenges AI confidence when the output feels too clean to be true
PL-07	External Review Cross-family review prompt structured for the VERIFY phase	PL-08	Drift Sweep Weekly RECONCILE pass across all five pairs

PL-09

Model Transition Audit

Per-model-release check that nothing depends on version-specific behaviour

Twelve Failure Modes

Named failures make the conversation possible. "That looks like FM-02 (Wrong-file Attribution)" is shorter than the long description — and communicates more precisely.

FM-01 Hallucinated API

AI invents methods that don't exist in the library

FM-02 Wrong-file Attribution

Fix applied to the file AI thought contained the bug

FM-03 Phantom Success

AI reports task complete; task is not complete

FM-04 Confident Fabrication

Clear, assertive description of something that isn't true

FM-05 Regression Introduction

Fix for bug A breaks previously-working feature B

FM-06 Scope Creep

AI rewrites adjacent code that wasn't part of the request

FM-07 Defensive Paranoia

AI adds null checks and try/catch where they're not needed

FM-08 Convention Drift

New code doesn't match the codebase's existing style

FM-09 Stale Context

AI operates on what the codebase looked like last week

FM-10 Silent Regeneration Loss

Regenerated document silently drops load-bearing sections

FM-11 Partial Migration

AI migrates some callers to a new API, not all

FM-12 Spec-Impl Gap

AI implements what it thinks the spec said, not what it says

Operational Cadence

CADENCE	WHAT HAPPENS	MECHANISM
Daily	Sealed contract hash check	CI; no human action unless hash changes
Weekly	RECONCILE sweep across five pairs	PL-08 prompt; 1-2 hours
Sprint-end	Compliance state review for the sprint's units	VERIFY outputs consolidated; 30 min
Milestone	Full VERIFY with zero-tolerance for open deltas	Not declared done until all pairs clean
Per model release	Experiment-first discipline; Model Matrix update	PL-09 transition audit before adoption
Quarterly	Team-level SRB review (team-autonomous); feeds enterprise SRB (full mode)	Contract integrity scores reviewed

Alignment with SDLC v2.0

MBG v1.1 was written for SDLC v1.1. The prompts, failure modes, and rituals all continue to work under SDLC v2.0, but reference some v1.1 vocabulary (SSOT, Feature Checklist) that v2.0 has superseded. An MBG v1.2 update aligning vocabulary is anticipated once real v2.0 adoption produces feedback.

↓ Download PDF

Read Case Study →

From Solo to Production

A Case Study

The twelve-month build that produced the methodology. A bilingual Android and PostgreSQL education platform for Indian Class 10 students, built almost entirely solo across five generations of AI models. The drift incidents CD1–CD4 documented in detail.

VERSION	AUDIENCE	LICENCE
1.1 · April 2026	Sceptics · Evaluators	CC BY 4.0

↓ Download PDF

↓ DOCX

Why This Case Study Remains Unchanged

The Case Study documents what actually happened during the MiVA build. It was written before SEA was formalised (the framework was first published as SEA 1.0 in April 2026 and revised to 1.1 in May 2026). Rewriting it now to frame events through SEA's vocabulary would falsify the chronology — the build was not conducted under SEA, it was the empirical basis from which SEA was later derived. SEA 1.1 Annex A provides the SEA-aware retrospective framing. The Case Study itself remains as historical documentation.

What The Build Demonstrated

Sealed contracts survive model transitions

Across five generations of AI models, the sealed API interface did not need modification.

Drift is real and traceable

Four specific incidents (CD1–CD4), each traceable to a specific missing control.

Intent capture is essential

Even partial intent capture — via an informal SSOT — was the difference between catching errors and shipping them.

Cross-family review works

When ChatGPT reviewed Claude-built code and vice versa, it caught failures same-family review missed.

The Four Drift Incidents

Each incident is documented with context, detection, root-cause analysis, and the rule it produced. Annex A of SEA 1.1 reframes each through the SEA principle it violated.

INCIDENT	WHAT HAPPENED	SEA PRINCIPLE VIOLATED
CD1	SSOT declared 19 sealed functions; code had 23	P1 · Contracts durable, implementations transient
CD2	SSOT regenerated; two sections silently lost	P2 · Intent captured, not just behaviour
CD3	Bug fix applied to wrong file; AI described instead of reading	P7 · AI-readability + Source-First
CD4	V1-V11 fixes applied but SSOT still showed "Pending"	Authority Chain · single source of truth

↓ Download Full Case Study

The Drift Audit

Methodology

A standalone two-week diagnostic for organisations using AI in production. Fixed-scope, fixed-fee, decision-ready. Four tiers plus a monthly Retainer. Five reconciliation pairs. Non-negotiable severity classification.

VERSION

1.1 · April 2026

AUDIENCE

CTOs · Engineering Leaders

TYPE

Commercial engagement

↓ Download PDF

Book a qualifying call

Four Engagement Tiers

TIER	DURATION	SCOPE	BEST FOR
Starter	5 working days	One reconciliation pair, one subsystem, up to 5 findings	Proof-of-value; small teams
Standard	10 working days	All five reconciliation pairs, one subsystem, unlimited findings	Most engagements · default offering
Extended	15 working days	All five pairs, multiple subsystems, interview-heavy	Larger organisations; multi-team
Retainer	1 day / month	Monthly Reconciliation Sweep using PL-08 methodology	Post-audit, high-velocity teams

Severity Classification

Every finding gets a non-negotiable severity. Consistent classification across engagements is what makes the audit a credible diagnostic rather than a subjective report.

Critical

Could cause production incident, security breach, or silent data loss if shipped.

High

Could cause visible bug, failed deployment, or incorrect business decision within 30 days.

Medium

Creates confusion, slows onboarding, or increases future drift risk without immediate failure.

Low

Cosmetic or stylistic inconsistencies with no operational impact.

When to Say No

The methodology includes explicit disqualifiers. Decline engagements that match them — they produce bad outcomes.

- Client expects the audit to produce working code changes, not a plan — that is a Remediation Sprint, priced differently
- Client expects the audit to exonerate or blame specific individuals — the methodology identifies process gaps, not people
- No specification exists and none is planned — without a spec, there is nothing to reconcile code against
- Client will not grant read access to code, SSOT, and decisions together — the audit cannot function without all three
- Timeline demand is under 5 working days — the methodology cannot compress further without loss of rigour

The qualifying call

A 30-minute qualifying call precedes every engagement. No commitment. The call establishes whether the codebase has a drift problem and whether a Drift Audit is the right response.

Book a qualifying call

30 minutes. No commitment. Find out whether your codebase has a drift problem — and whether an audit is the right response.

[Email Suneesh](#)[LinkedIn ↗](#)[GitHub ↗](#)

Three phases. *Team to enterprise.*

Adopting the full seven-document methodology is not a single decision. It is a journey with three natural phases, each valuable on its own, each a platform for the next. Start where it makes sense. Stop when the value is enough. Continue when the next phase becomes warranted.

PHASE 1

Pilot · 2-4 weeks

PHASE 2

Scale · 3-6 months

PHASE 3

Mature · 12+ months

— WHO THIS IS FOR

You already have a *drift problem*.

If your team uses AI coding assistants and has said any of the following, you are already experiencing drift. You're not choosing whether to address it — you're choosing whether to address it deliberately.

Symptom

"It worked in the last version — I don't know why it broke."

Symptom

"The fix looked right — the bug is still there."

Symptom

"Nobody is sure which document is the source of truth."

Symptom

"The AI said it was fixed. It wasn't."

Why existing tools miss this

CI/CD checks code against tests. Code reviews check deltas, not absolute state. PR gates check changes, not consistency. None were designed for a world where artefacts are authored by different AI sessions at different times — each plausible on its own, none reconciled with each other.

— THE THREE PHASES

A journey, not a *big-bang adoption*.

Each phase is complete on its own. Teams can stay in Phase 1 indefinitely. Phase 2 becomes warranted when the team grows or the organisation commits. Phase 3 is for enterprises that genuinely need long-lived architectural governance.

1

PHASE 1 · PILOT

Team-autonomous SDLC v2.0

Duration · 2-4 weeks to functional · ongoing

Your team adopts SDLC v2.0 in team-autonomous mode. No enterprise SEA commitment required. No org restructure. Your team's senior members act as Contract Owner and Reviewer. The team itself is the quarterly review body. You produce SEA-native artefacts (sealed contracts, intent documents) without needing anyone outside the team to change how they work.

WHAT YOU'LL HAVE AFTER PHASE 1

- ▶ 3-6 sealed contracts for your team's most critical interfaces
- ▶ Weekly RECONCILE discipline catching drift as it happens
- ▶ Cross-family AI review producing fewer CD-series incidents
- ▶ Measurable Contract Integrity Scores per contract

2

PHASE 2 · SCALE

Add SEA 1.1 at enterprise

Duration · 3-6 months typical · when scale warrants

Your organisation commits to SEA at enterprise level. A Sealed Review Board is constituted. Contract Owners become formally named roles across teams. Team-level contracts propose upward into the enterprise registry. Governance rhythms align. Your team's existing work transitions into full mode; other teams start from scratch or from their own Phase 1.

WHAT YOU'LL HAVE AFTER PHASE 2

- ▶ Enterprise Sealed Review Board operating on quarterly cadence
- ▶ Cross-team sealed contracts with named Owners
- ▶ Enterprise drift incident log feeding governance decisions
- ▶ Portfolio-level integrity metrics visible to leadership

3

PHASE 3 · MATURE

Full Handbook operational

Duration · 12+ months after Phase 2 · multi-year

The full Implementation Handbook becomes operational. Build-pipeline enforcement with hash verification and CODEOWNERS. Runtime verification for invariants. Five compliance states with severity classification. Extension pattern catalogue applied. First parallel-live replacement executed. Rollback protocol tested. Governance health indicators tracked across the portfolio.

WHAT YOU'LL HAVE AFTER PHASE 3

- ▶ Hash-verified sealed contracts; cryptographic audit trail
- ▶ Layered enforcement across repository, build, runtime, governance
- ▶ Successful executed evolution events (extension, parallel-live)
- ▶ Systems demonstrably more resilient to model, team, and purpose change

Which phase is *right* for you?

YOUR SITUATION	START AT	WHY
Team of 5-15, using AI daily, experiencing drift symptoms	Phase 1	Immediate value, minimal commitment, no organisational dependency
Team is part of an organisation actively considering SEA	Phase 1 → Phase 2 within 6 months	Phase 1 validates the approach; Phase 2 becomes the natural next step
CTO / VP Engineering exploring enterprise adoption	Drift Audit first, then Phase 2	Audit surfaces whether adoption is justified and where to start
Regulated enterprise with long-lived systems	Phase 2 + Phase 3 in parallel	Scale needs enterprise governance; regulation justifies full Handbook
Solo developer or pre-product team	Read SEA 1.1; don't adopt yet	Premature adoption produces ceremony without value; design for it later

Not sure where to start?

A 30-minute conversation can map your situation against the three phases and recommend an entry point. Often the answer is a Drift Audit first, then Phase 1, then a decision on Phase 2 based on what the audit surfaced.

[Book a conversation](#)[Learn about Drift Audit](#)

We don't replace TOGAF.
We don't replace *DDD*.
We don't replace *Scrum*.

We add the governance layer those frameworks have historically assumed exists. Each framework is good at what it does. None specifies cross-artefact integrity for AI-authored work. None has a formal governance protocol for long-lived architectural artefacts. That's the gap SEA and the AI-Native SDLC fill — and they fill it alongside your existing frameworks, not instead of them.

The honest positioning

Enterprise architecture has TOGAF. Domain modelling has DDD. Code architecture has Clean and Hexagonal. Team process has Scrum and SAFe. Service architecture has Microservices patterns. AI-augmented coding is beginning to have Spec-Driven Development. Every one of these is good, well-established, and widely adopted.

But none of them answer two specific questions that matter in 2026:

1. **How does the architecture survive when the AI models authoring it change every six months?** Every established framework assumes a stable human author. Stable AI author is not a property of current AI, and won't be for the foreseeable future.
2. **How do the durable decisions survive the next decade of changes in team, ownership, purpose, and technology?** Architectural frameworks describe static structure well. Almost none specify how structure evolves across generations of custodians.

SEA 1.1 addresses these questions explicitly. The AI-Native SDLC v2.0 operationalises the answers at team level. Neither framework competes with TOGAF or DDD or Scrum — they complement and extend what those frameworks already do.

FRAMEWORK BY FRAMEWORK

What each does well. What each *doesn't address*. How we complement it.

Six frameworks you likely already use. For each: honest appreciation of what it does well, specific acknowledgement of what falls outside its scope, and concrete mapping of how SEA and SDLC v2.0 integrate without displacing.

T

TOGAF

ENTERPRISE ARCHITECTURE FRAMEWORK

✓ WHAT TOGAF DOES WELL

The Architecture Development Method. The Architecture Continuum. Structured enterprise architecture governance at scale, with well-defined roles, artefacts, and review gates. Decades of practitioner experience.

✗ WHAT TOGAF DOESN'T ADDRESS

TOGAF describes the structure of enterprise architecture, not the lifecycle of individual architectural artefacts. It has no specific mechanism for AI-authored work. No hash verification, no sealed contracts, no cross-artefact reconciliation pairs.

+ HOW SEA COMPLEMENTS

Sealed contracts become first-class elements of the TOGAF Architecture Continuum. The Sealed Review Board aligns with TOGAF's Architecture Board. SDLC v2.0 phases map to ADM development phases. SEA fills the artefact-lifecycle gap TOGAF intentionally leaves open.

D**Domain-Driven Design**

DOMAIN MODELLING APPROACH

✓ WHAT DDD DOES WELL

Aggregate roots, bounded contexts, ubiquitous language. A vocabulary for expressing domain structure that survives technology change. Strategic patterns (context mapping) for organisational-scale modelling.

✗ WHAT DDD DOESN'T ADDRESS

DDD gives you the shape of good domain boundaries but not the formal mechanism for sealing them. No specification of how aggregate roots are versioned, governed, or succeeded. No built-in protocol for extension vs replacement.

+ HOW SEA COMPLEMENTS

Aggregate roots map directly to sealed contract boundaries. Bounded contexts become governance domains with their own SRBs. SEA's evolution mechanisms (extension, parallel-live, MVB) give DDD-modelled systems the governance they otherwise lack.

C**Clean / Hexagonal Architecture**

CODE-LEVEL ARCHITECTURE

✓ WHAT CLEAN/HEXAGONAL DOES WELL

Ports and adapters. Dependency inversion. Clean separation between business logic and infrastructure. Code that tolerates technology change because infrastructure is replaceable.

✗ WHAT CLEAN/HEXAGONAL DOESN'T ADDRESS

Clean tells you how to structure code for replaceability, but not how to formally seal the port definitions. Ports are conventions, not contracts. Nothing prevents casual modification, hash-verifies conformance, or governs evolution.

+ HOW SEA COMPLEMENTS

Ports are SEA sealed contracts. Adapters are transient implementations. The Clean conceptual structure stays identical; SEA adds formal sealing, intent capture, and governance around the ports you already have.

S

Scrum / SAFe

TEAM PROCESS FRAMEWORKS

✓ WHAT SCRUM/SAFE DOES WELL

Sprint cadence. Backlog management. Iterative delivery. Scaled coordination across teams (SAFe). Well-understood ceremonies and metrics. Good at velocity, good at responding to change.

✗ WHAT SCRUM/SAFE DOESN'T ADDRESS

Process frameworks describe how teams work, not what they produce. No specification of architectural artefact lifecycle. No protocol for AI-authored work. No mechanism for catching drift between what the team thinks it built and what it actually shipped.

+ HOW SDLC V2.0 COMPLEMENTS

SDLC v2.0 phases fit inside sprint cadence. Extension protocol fits sprint planning. RECONCILE runs at sprint boundary. Quarterly SRB reviews align with SAFe's Inspect & Adapt. Your Scrum doesn't change; v2.0 adds the architectural discipline Scrum assumes you bring.

μ

Microservices Architecture

SERVICE ARCHITECTURE

✓ WHAT MICROSERVICES DOES WELL

Service boundaries. Independent deployability. Team autonomy around services. Containerisation, orchestration, and service mesh infrastructure. Well-established patterns for inter-service communication.

✗ WHAT MICROSERVICES DOESN'T ADDRESS

Microservices gives you services that can evolve independently, but not the governance that ensures their public APIs evolve safely. No formal mechanism for API versioning discipline, consumer tracking, or cross-service invariant enforcement.

+ HOW SEA COMPLEMENTS

Each service's public API is a sealed contract. Service internals remain transient. Consumer registry is SEA-native. Extension and parallel-live protocols become the mechanism for API evolution that the microservices community has long debated.



Spec-Driven Development

AI-ERA DEVELOPMENT APPROACH

✓ WHAT SDD DOES WELL

Specifications as the primary artefact. AI generates implementation from specs. Specs are version-controlled, reviewable, and human-readable. A major step forward for AI-augmented development.

x WHAT SDD DOESN'T ADDRESS

SDD generates specs but does not distinguish between transient specs and durable ones. No concept of sealed contracts vs extension proposals. No governance protocol for who can modify which specs and under what authority.

+ HOW SEA COMPLEMENTS

SDD and SEA are complementary. SDD generates broad specifications; SEA seals the critical subset. The sealed contracts become the governance-protected core around which SDD continues to generate flexible specifications. Use both.

One-table summary

FRAMEWORK	WHAT IT COVERS	WHAT THE METHODOLOGY ADDS
TOGAF	Enterprise architecture structure and governance	Per-artefact lifecycle, sealing, AI-era evolution mechanisms
DDD	Domain modelling and bounded contexts	Formal sealing of aggregate boundaries; governance of context maps
Clean / Hexagonal	Code-level separation of concerns	Sealed port definitions with hash verification
Scrum / SAgE	Team and scaled team process	Artefact lifecycle phases that fit inside sprint cadence

FRAMEWORK	WHAT IT COVERS	WHAT THE METHODOLOGY ADDS
Microservices	Service-boundary architecture	Sealed public APIs with consumer tracking and evolution protocols
Spec-Driven Development	AI-generated specifications as primary artefact	Distinction between transient and sealed specs; governance protocol

Adoption recipe — Scrum team already using DDD

Concretely, how does a mid-sized engineering team already running Scrum with DDD patterns integrate SEA and SDLC v2.0 without replacing anything?

1. **Keep your Scrum cadence.** Your sprints, standups, retros, and planning meetings continue unchanged. SDLC v2.0 phases happen within the work already scheduled, not in parallel to it.
2. **Identify your aggregate roots.** Your DDD modelling has already surfaced the natural sealing boundaries. Each aggregate root is a candidate for a sealed contract. Pick 3-5 most critical ones for Phase 1.
3. **Adopt team-autonomous mode first.** Your team becomes its own Sealed Review Board. Contract Owner and Reviewer are senior members of the team. No enterprise commitment required.
4. **Add the RECONCILE weekly sweep.** One hour per week applying PL-08 across the five reconciliation pairs for your sealed contracts. Finds drift within days instead of months.
5. **Use SDLC v2.0's VERIFY with your existing QA process.** The five compliance states complement — not replace — your existing test suite. QA catches behavioural defects; VERIFY catches architectural drift.
6. **Escalate to enterprise SEA only when warranted.** If the organisation decides to adopt at enterprise level, your Phase 1 work transitions to Phase 2 without rework — you were already producing SEA-compatible artefacts.

The meta-point

If a framework you already use is not in this list, it probably still applies. SEA and SDLC v2.0 are not domain-specific — they work with any existing architectural or process framework that leaves artefact lifecycle as an open question. Which is most of them.

READY TO INTEGRATE?

Start with the Drift Audit to surface where your existing frameworks have gaps. Then adopt Phase 1 of the journey to fill them. No existing framework has to change.

[See the adoption journey →](#)[Book a Drift Audit](#)

The AI-Native SDLC

v1.1 · *Archived*

The pre-SEA SDLC. Superseded by v2.0 in April 2026 via formal Major Version Bump. Preserved as reference for teams currently operating on v1.1 who have not yet migrated. The archival itself demonstrates SEA's retirement-with-dignity principle — history preserved, not deleted.

VERSION	STATUS	SUPERSEDED BY
1.1 · April 2026	Archived	SDLC v2.0

[↓ Download v1.1 PDF](#)[Go to v2.0 →](#)

Why v1.1 Was Archived, Not Retired

SEA 1.0 specifies that sealed artefacts end with dignity. When a framework reaches the end of its current applicability, it is archived — its specification preserved, its history intact, its consumers given time to migrate — not simply deleted.

v1.1 was archived when v2.0 was published because its premises had been superseded, not because it was broken. The framework continues to work for teams currently using it. Teams should not feel pressured to migrate; v1.1 will not be updated further, but it remains a valid and operational framework for the audiences and situations it was designed to serve.

Should You Stay on v1.1?

Situations where staying on v1.1 is the right choice:

- Small team (under 5 engineers) with a short product horizon (under 2 years)
- Organisation has no plan to adopt SEA and the team is not positioned to lead the adoption

- Team is mid-way through a major v1.1 implementation (complete it first, then consider migration)
- Team is fatigued by process changes — imposing v2.0 on a change-fatigued team produces theatre, not discipline

What v1.1 Contains

The 7-phase lifecycle (FRAME → SEAL → SPEC → BUILD → VERIFY → RECONCILE → EVOLVE), the load-bearing "Ready for external review" phrase, the one-chat-per-unit rule, Source-First enforcement, cross-family AI review, RACI with AI as a role, and the two-level VERIFY structure (Full vs Lite). All of these transfer to v2.0 largely unchanged. What v1.1 lacks — that v2.0 provides — is SEA-native artefacts, explicit governance structure, and the team-to-enterprise interface.

Migration path to v2.0

When ready, the typical migration takes one to two sprints. SDLC v2.0 Part V specifies the path in detail: pre-migration preparation, a focused migration sprint with day-by-day breakdown, and the first-quarter post-migration observations.

[↓ Download v1.1 PDF](#)[Read v2.0 →](#)

Sealed Enterprise

Architecture 1.0 · Archived

The original SEA whitepaper, published April 2026. Superseded by SEA 1.1 in May 2026 via a focused revision (not a rewrite). Preserved as the historical record of the framework's first articulation. The archival itself demonstrates SEA's retirement-with-dignity principle — history preserved, not deleted.

VERSION	STATUS	SUPERSEDED BY
1.0 · April 2026	Archived	SEA 1.1

[↓ Download v1.0 PDF](#)[Go to v1.1 →](#)

Why v1.0 Was Archived, Not Retired

SEA 1.0 specifies that sealed artefacts end with dignity. When a document reaches the end of its current applicability, it is archived — its specification preserved, its history intact, its readers given clear pointers to the successor — not simply deleted or quietly overwritten. SEA 1.1 applies that principle to its own predecessor.

v1.0 was archived when v1.1 was published because the framing and completeness of the original could be strengthened, not because the substance was wrong. The seven principles, the 14 failure modes, the five layers, the 32 gaps, the sealed contract format, the governance protocol structure, and the enterprise adoption pathway are unchanged between 1.0 and 1.1. What changed is positioning, tone, and completeness — see the change history in SEA 1.1 for the specific diffs.

What Changed in 1.0 → 1.1

SEA 1.1 is a focused revision, not a rewrite. Roughly 85 percent of the prose is unchanged. The seven targeted changes were:

- Subtitle: *"An Architectural Philosophy for Systems That Must Outlive Everything About Them"* → *"A Governance and Continuity Framework for Long-Lived AI-Era Systems"* — signals more accurately that SEA addresses a longevity-and-governance problem alongside existing architectural frameworks rather than replacing them.
- New "What SEA Is Not" subsection in the Executive Summary — explicit disclaimer that SEA does not replace TOGAF, DDD, Clean, microservices, or Scrum/SAFe.
- Tone calibration on Section 1's "no such discipline" claim — acknowledges Parnas's information hiding, bounded contexts, ports and adapters, ADRs, and semantic versioning as fragmentary precursors. The discipline is not consolidated, not absent.
- New Section 12.9: "Why SEA Does Not Become Bureaucracy" — names six design choices that prevent the slide into governance theatre.
- New Annex A.6: "Hypothetical Applications" — three thought experiments (auth provider replacement, LLM provider migration, M&A continuity), explicitly labelled as thought experiments not case studies.
- Companion documents updated from "five" to "seven" — current methodology set.
- Change history block at end of document — itself an SEA principle applied.

Should You Read v1.0 Instead of v1.1?

Almost certainly not. v1.1 contains everything v1.0 contains, plus the additions above. The only legitimate reasons to consult v1.0 are: comparing the two versions side-by-side for research; citing the original publication date (April 2026) for historical purposes; or verifying that v1.1's claim of "85 percent unchanged" is accurate. For all other purposes, v1.1 is the canonical reading.

SEA's discipline applied to itself

Preserving v1.0 alongside v1.1 is not nostalgia; it is the framework being honest about its own evolution. If SEA cannot survive a focused revision without losing the previous version's authority, it cannot ask consumers to trust it with their own long-lived systems. The archival is part of the demonstration.

↓ Download v1.0 PDF

Read v1.1 →

Suneesh Sidharthan

Solution Architect · IT Project Manager · AI Platform Engineering. Available for Drift Audit engagements, methodology consulting, and SEA adoption support.

suneesh.sidharthan80@gmail.com

+91 9986898076

Bengaluru, Karnataka, India

FOUNDATION

SEA 1.1

Implementation Handbook

OPERATIONS

SDLC v2.0

Master Build Guide

SUPPORTING

Case Study

Drift Audit

vs Industry

Adoption Journey

SDLC v1.1 (archive)

SEA 1.0 (archive)

CONNECT

Email

LinkedIn ↗

GitHub ↗

© 2026 Suneesh Sidharthan · All documents released under [CC BY 4.0](#)

Seven documents · Three tiers · One methodology